

Završni projekt iz informatike

Kartaska igra inspirirana igrom *Slay the Spire*

Izradjeno u: **Godot Engine 4**

Baza podataka: **SQLite**

16. svibnja 2026.

Sadržaj

1	Opis projekta	2
1.1	Tehnologije	2
2	Struktura projekta	2
3	Baza podataka	2
3.1	Shema baze podataka	2
3.2	Opis tablica	3
3.3	Unos podataka	3
4	GDScript skripte	4
4.1	Card.gd	4
4.2	Player.gd	4
4.3	Protivnik.gd	5
4.4	Main.gd	5
4.4.1	Varijable	5
4.4.2	Pokretanje i inicijalizacija	6
4.4.3	Game loop – faza igrača	7
4.4.4	Game loop – faza protivnika	8
4.4.5	Round sistem i nagrade	9
4.4.6	Timer	10
4.4.7	Učitavanje iz baze podataka	10
5	GUI struktura	12
6	Tijek igre	12
7	Zaključak	13

Opis projekta

Ovaj projekt je kartaska igra za jednog igrača izrađjana u **Godot Engine 4**. Igra je inspirirana popularnom roguelike deck-building igrom *Slay the Spire*. Cilj igre je poraziti tri protivnika redom – Goblina, Viteza i Zmaja – koristeći karte iz svog spila. Igra je povezana s **SQLite** bazom podataka iz koje se dinamički učitavaju svi podaci o korisnicima, kartama, spilovima i protivnicima. Nakon svake pobjede igrač dobiva nagradu, a cijela igra prati ukupno vrijeme igranja.

Tehnologije

- **Godot Engine 4** – game engine i skriptiranje (GDScript)
- **SQLite** (GDSQLite plugin) – relacijska baza podataka
- **GDScript** – programski jezik specifičan za Godot
- **GitHub** – verzioniranje i pohrana koda

Struktura projekta

Datoteka	Opis
Scripts/Card.gd	Resource klasa koja predstavlja jednu kartu
Scripts/Player.gd	Klasa koja predstavlja igrača
Scripts/Protivnik.gd	Klasa koja predstavlja protivnika
Scripts/Main.gd	Glavna skripta – sav GUI i logika igre
game.db	SQLite baza podataka
addons/	GDSQLite plugin

Baza podataka

Shema baze podataka

```
1 CREATE TABLE korisnik (  
2     id          INTEGER PRIMARY KEY AUTOINCREMENT ,  
3     username    TEXT ,  
4     email       TEXT  
5 );  
6  
7 CREATE TABLE protivnik (  
8     id          INTEGER PRIMARY KEY AUTOINCREMENT ,  
9     naziv       TEXT ,  
10    hp          INTEGER ,  
11    difficulty   TEXT  
12 );  
13  
14 CREATE TABLE karta (  
15     id          INTEGER PRIMARY KEY AUTOINCREMENT ,  
16     naziv       TEXT ,  
17     attack     INTEGER ,
```

```

18     cost    INTEGER
19 );
20
21 CREATE TABLE deck (
22     id        INTEGER PRIMARY KEY AUTOINCREMENT,
23     naziv     TEXT,
24     vlasnik_tip TEXT,
25     korisnik_id INTEGER,
26     protivnik_id INTEGER
27 );
28
29 CREATE TABLE deck_karta (
30     id        INTEGER PRIMARY KEY AUTOINCREMENT,
31     deck_id   INTEGER,
32     karta_id  INTEGER,
33     kolicina  INTEGER,
34     FOREIGN KEY(deck_id) REFERENCES deck(id),
35     FOREIGN KEY(karta_id) REFERENCES karta(id)
36 );

```

Listing 1: Kreiranje tablica

Opis tablica

- **korisnik** – podatci o igraču (korisnicko ime i email)
- **protivnik** – protivnici s HP-om i tezinom (easy/medium/hard)
- **karta** – sve dostupne karte s napadom i cijenom energije
- **deck** – spil koji pripada korisniku ili protivniku
- **deck_karta** – vezna tablica; definira karte i kolicine u spilu

Unos podataka

```

1 INSERT INTO korisnik (username, email) VALUES
2     ('Ivan', 'ivan@gmail.com');
3
4 INSERT INTO protivnik (naziv, hp, difficulty) VALUES
5     ('Goblin', 20, 'easy'),
6     ('Vitez', 35, 'medium'),
7     ('Zmaj', 50, 'hard');
8
9 INSERT INTO karta (naziv, attack, cost) VALUES
10     ('Mac', 5, 1),
11     ('Vatrogasna', 8, 2),
12     ('Munja', 12, 3),
13     ('Udarac', 3, 1),
14     ('Eksplozija', 15, 3),
15     ('Strijela', 6, 2);
16
17 -- Player deck

```

```

18 INSERT INTO deck (naziv, vlasnik_tip, korisnik_id, protivnik_id)
19     VALUES ('Player Deck', 'korisnik', 1, NULL);
20
21 INSERT INTO deck_karta (deck_id, karta_id, kolicina) VALUES
22     (1, 1, 2), (1, 2, 2), (1, 3, 2),
23     (1, 4, 2), (1, 5, 1), (1, 6, 2);
24
25 -- Goblin deck
26 INSERT INTO deck (naziv, vlasnik_tip, korisnik_id, protivnik_id)
27     VALUES ('Goblin Deck', 'protivnik', NULL, 1);
28
29 INSERT INTO deck_karta (deck_id, karta_id, kolicina) VALUES
30     (2, 1, 3), (2, 4, 3), (2, 6, 2);
31
32 -- Vitez deck
33 INSERT INTO deck (naziv, vlasnik_tip, korisnik_id, protivnik_id)
34     VALUES ('Vitez Deck', 'protivnik', NULL, 2);
35
36 INSERT INTO deck_karta (deck_id, karta_id, kolicina) VALUES
37     (3, 1, 2), (3, 2, 3), (3, 4, 2), (3, 6, 3);
38
39 -- Zmaj deck
40 INSERT INTO deck (naziv, vlasnik_tip, korisnik_id, protivnik_id)
41     VALUES ('Zmaj Deck', 'protivnik', NULL, 3);
42
43 INSERT INTO deck_karta (deck_id, karta_id, kolicina) VALUES
44     (4, 2, 3), (4, 3, 3), (4, 5, 2), (4, 6, 2);

```

Listing 2: INSERT naredbe

GDScript skripte

Card.gd

Klasa Card nasljedjuje Resource. Svaka karta ima jedinstveni identifikator, naziv, vrijednost napada i cijenu energije.

```

1 extends Resource
2 class_name Card
3
4 var id      : int
5 var naziv   : String
6 var attack  : int
7 var cost    : int

```

Listing 3: Card.gd

Player.gd

Klasa Player nasljedjuje Node. Metoda draw_card() izvlaci random kartu iz spila u ruku.

```

1 extends Node

```

```

2  class_name Player
3
4  var hp      : int    = 20
5  var energy  : int    = 3
6  var deck    : Array = []
7  var hand    : Array = []
8
9  func draw_card():
10     if deck.size() > 0:
11         var random_index = randi() % deck.size()
12         var card = deck[random_index]
13         deck.remove_at(random_index)
14         hand.append(card)

```

Listing 4: Player.gd

Protivnik.gd

Klasa `Enemy` nasljedjuje `Node`. Sadrzi iste attribute kao `Player` uz dodatak polja `difficulty`.

```

1  extends Node
2  class_name Enemy
3
4  var hp      : int    = 20
5  var energy  : int    = 3
6  var difficulty : String = "easy"
7  var deck    : Array = []
8  var hand    : Array = []
9
10 func draw_card():
11     if deck.size() > 0:
12         var random_index = randi() % deck.size()
13         var card = deck[random_index]
14         deck.remove_at(random_index)
15         hand.append(card)

```

Listing 5: Protivnik.gd

Main.gd

Glavna skripta upravlja cjelokupnom logikom igre, GUI-em i bazom podataka. Sav GUI gradi se dinamički kroz kod – nema zasebnih scene datoteka.

4.4.1 Varijable

```

1  extends Node
2
3  # Podaci igre
4  var player      : Player
5  var enemy       : Enemy
6  var game_over   = false
7  var player_turn = true

```

```

8  var waiting_for_input = false
9  var player_username   = "Igrac"
10 var player_email      = ""
11 var first_card_played = false
12 var current_round     = 1
13 var max_rounds        = 3
14 var player_max_hp     = 20
15 var player_max_energy = 3
16 var enemy_name        = ""
17 var enemy_difficulty  = ""
18
19 # Timer
20 var elapsed_seconds    = 0
21 var timer_label       : Label
22 var game_timer        : Timer
23
24 # Reward gumbi
25 var reward_hp_btn     : Button
26 var reward_energy_btn : Button
27 var reward_chosen     = false
28
29 # End screen labeli
30 var end_msg_label     : Label
31 var time_msg_label    : Label
32
33 # Protivnici po roundu
34 var round_enemies = {
35     1: {"naziv": "Goblin", "difficulty": "easy"},
36     2: {"naziv": "Vitez",  "difficulty": "medium"},
37     3: {"naziv": "Zmaj",   "difficulty": "hard"},
38 }

```

Listing 6: Main.gd – varijable

4.4.2 Pokretanje i inicijalizacija

```

1  func _ready():
2      _copy_db()
3      _build_menu()
4      _build_about()
5      _build_game()
6      _build_reward()
7      _build_end_screen()
8
9      menu_window.visible = true
10     about_window.visible = false
11     game_window.visible = false
12     reward_window.visible = false
13     end_window.visible = false
14
15 func _copy_db():
16     if not FileAccess.file_exists("user://game.db"):

```

```

17         DirAccess.copy_absolute("res://game.db", "user://game.db"
18         )
19
20 func start_game():
21     player = Player.new()
22     enemy = Enemy.new()
23     game_over = false
24     first_card_played = false
25     player.hp = player_max_hp
26
27     load_player_from_database()
28     load_cards_for_round()
29     load_enemy_for_round()
30
31     for i in range(3):
32         player.draw_card()
33         enemy.draw_card()
34
35     round_label.text = "Round " + str(current_round) + " / 3"
36
37     enemy_name_label.text = enemy_name
38     enemy_diff_label.text = enemy_difficulty.to_upper()
39
40     match enemy_difficulty:
41         "easy": enemy_diff_label.add_theme_color_override(
42             "font_color", Color(0.20, 0.85, 0.20))
43         "medium": enemy_diff_label.add_theme_color_override(
44             "font_color", Color(0.30, 0.60, 1.00))
45         "hard": enemy_diff_label.add_theme_color_override(
46             "font_color", Color(1.00, 0.25, 0.25))
47
48     log_label.text = ""
49     add_log("Dobrodosao, " + player_username +
50         "! Round " + str(current_round) + " pocinje.")
51     update_hud()
52     player_phase()

```

Listing 7: Main.gd – `_ready()` i `start_game()`

4.4.3 Game loop – faza igraca

Na pocetku svakog poteza igrac dobiva energiju jednaku `player_max_energy` i izvlaci random kartu iz spila. Potez se automatski prebacuje na protivnika ako igrac nema energije, nema karata ili ne moze platiti nijednu kartu.

```

1 func player_phase():
2     player.energy = player_max_energy
3     player.draw_card()
4     player_turn = true
5
6     set_status("Tvoj potez, " + player_username + "!")
7     update_hud()

```



```

8
9     var can_play = refresh_hand()
10
11     if player.hand.size() == 0:
12         add_log("Nemas karata u ruci.")
13         await get_tree().create_timer(0.8).timeout
14         end_player_turn()
15         return
16
17     if not can_play:
18         add_log("Ne mozes platiti nijednu kartu -- potez
19             automatski prelazi.")
20         waiting_for_input = false
21         await get_tree().create_timer(1.0).timeout
22         end_player_turn()
23         return
24
25     waiting_for_input = true

```

Listing 8: Main.gd – player_phase()

4.4.4 Game loop – faza protivnika

Protivnik automatski igra sve karte koje moze platiti energijom. Zmaj ima 4 energije, ostali protivnici imaju 3.

```

1 func enemy_phase():
2     set_status("Protivnikov potez...")
3     player_turn = false
4
5     match enemy_name:
6         "Zmaj": enemy.energy = 4
7         _:      enemy.energy = 3
8
9     enemy.draw_card()
10    update_hud()
11    await get_tree().create_timer(0.8).timeout
12
13    var played_any = true
14    while played_any and enemy.hand.size() > 0 and enemy.energy >
15        0:
16        played_any = false
17        for i in range(enemy.hand.size()):
18            var card = enemy.hand[i]
19            if card.cost <= enemy.energy:
20                enemy.hand.remove_at(i)
21                enemy.energy -= card.cost
22                enemy_attack(card)
23                add_log("Enemy igra: " + card.naziv +
24                    " [ATK: " + str(card.attack) +
25                    " COST: " + str(card.cost) + "]")
26                update_hud()

```

```

26         played_any = true
27         await get_tree().create_timer(0.6).timeout
28         break
29
30     if await check_game_over():
31         return
32     player_phase()

```

Listing 9: Main.gd – enemy_phase()

4.4.5 Round sistem i nagrade

Nakon pobjede nad protivnikom otvara se ekran s nagradama. Igrac moze odabrati jednu od dvije nagrade – nagrada se moze odabrati samo jednom, a gumbi se zasive nakon odabira.

```

1 func _on_reward_hp():
2     if reward_chosen:
3         return
4     reward_chosen = true
5     player_max_hp += 10
6     add_log("Nagrada: +10 max HP! (novi max: " + str(
7         player_max_hp) + ")")
8     _disable_reward_buttons()
9
10 func _on_reward_energy():
11     if reward_chosen:
12         return
13     reward_chosen = true
14     player_max_energy += 1
15     add_log("Nagrada: +1 max energija! (novi max: "
16         + str(player_max_energy) + ")")
17     _disable_reward_buttons()
18
19 func _on_next_round():
20     reward_window.visible = false
21     reward_chosen = false
22     reward_hp_btn.disabled = false
23     reward_energy_btn.disabled = false
24     current_round += 1
25     start_game()
26
27 func _on_restart():
28     end_window.visible = false
29     game_window.visible = true
30     current_round = 1
31     player_max_hp = 20
32     player_max_energy = 3
33     elapsed_seconds = 0
34     timer_label.text = "00:00"
35     _start_timer()
36     start_game()

```

Listing 10: Main.gd – nagrade i roundovi

4.4.6 Timer

Timer prikazuje ukupno vrijeme igranja u gornjem desnom kutu u formatu MM:SS. Počinje kad igrač klikne "Pokreni igru" i zaustavlja se na kraju igre.

```
1 func _start_timer():
2     elapsed_seconds = 0
3     if game_timer:
4         game_timer.queue_free()
5     game_timer = Timer.new()
6     game_timer.wait_time = 1.0
7     game_timer.autostart = true
8     game_timer.timeout.connect(_on_timer_tick)
9     add_child(game_timer)
10
11 func _on_timer_tick():
12     elapsed_seconds += 1
13     var mins = elapsed_seconds / 60
14     var secs = elapsed_seconds % 60
15     timer_label.text = "%02d:%02d" % [mins, secs]
16
17 func _stop_timer():
18     if game_timer:
19         game_timer.stop()
```

Listing 11: Main.gd – timer

4.4.7 Ucitavanje iz baze podataka

```
1 func load_player_from_database():
2     var db = SQLite.new()
3     db.path = "user://game.db"
4     db.open_db()
5     var result = db.select_rows("korisnik", "", ["id", "username",
6         "email"])
7     if result.size() > 0:
8         player_username = result[0]["username"]
9         player_email = result[0]["email"]
10    db.close_db()
11
12 func load_cards_for_round():
13     var db = SQLite.new()
14     db.path = "user://game.db"
15     db.open_db()
16     var deck_result = db.select_rows(
17         "deck",
18         "vlasnik_tip = 'korisnik' AND korisnik_id = 1",
19         ["id"])
```

```

19     if deck_result.size() == 0:
20         db.close_db()
21         return
22     var deck_id = deck_result[0]["id"]
23     var dk = db.select_rows(
24         "deck_karta",
25         "deck_id = " + str(deck_id),
26         ["karta_id", "kolicina"])
27     for dk_row in dk:
28         var rows = db.select_rows(
29             "karta", "id = " + str(dk_row["karta_id"]),
30             ["id", "naziv", "attack", "cost"])
31         if rows.size() > 0:
32             for j in range(dk_row["kolicina"]):
33                 var card = Card.new()
34                 card.id = rows[0]["id"]
35                 card.naziv = rows[0]["naziv"]
36                 card.attack = rows[0]["attack"]
37                 card.cost = rows[0]["cost"]
38                 player.deck.append(card)
39     db.close_db()
40
41 func load_enemy_for_round():
42     var db = SQLite.new()
43     db.path = "user://game.db"
44     db.open_db()
45     var round_info = round_enemies[current_round]
46     enemy_name = round_info["naziv"]
47     enemy_difficulty = round_info["difficulty"]
48     var result = db.select_rows(
49         "protivnik",
50         "naziv = '" + enemy_name + "'",
51         ["id", "naziv", "hp", "difficulty"])
52     var enemy_id = 1
53     if result.size() > 0:
54         enemy.hp = result[0]["hp"]
55         enemy.difficulty = result[0]["difficulty"]
56         enemy_id = result[0]["id"]
57     var deck_result = db.select_rows(
58         "deck",
59         "vlasnik_tip = 'protivnik' AND protivnik_id = " + str(
60             enemy_id),
61         ["id"])
62     if deck_result.size() == 0:
63         db.close_db()
64         return
65     var deck_id = deck_result[0]["id"]
66     var dk = db.select_rows(
67         "deck_karta",
68         "deck_id = " + str(deck_id),
69         ["karta_id", "kolicina"])

```

```

69     for dk_row in dk:
70         var rows = db.select_rows(
71             "karta", "id = " + str(dk_row["karta_id"]),
72             ["id", "naziv", "attack", "cost"])
73         if rows.size() > 0:
74             for j in range(dk_row["kolicina"]):
75                 var card = Card.new()
76                 card.id = rows[0]["id"]
77                 card.naziv = rows[0]["naziv"]
78                 card.attack = rows[0]["attack"]
79                 card.cost = rows[0]["cost"]
80                 enemy.deck.append(card)
81 db.close_db()

```

Listing 12: Main.gd – load funkcije

GUI struktura

Cijeli GUI gradi se dinamički kroz kod u Main.gd bez zasebnih scene datoteka. Sastoji se od pet prozora koji se naizmjenično prikazuju ovisno o stanju igre.

Prozor	Varijabla	Opis
Glavni izbornik	menu_window	Pocetni ekran s gumbima
O igri	about_window	Informacije o igri
Igra	game_window	Glavni game ekran
Nagrada	reward_window	Izbor nagrade nakon pobjede
Kraj igre	end_window	Ekran pobjede ili poraza

Tijek igre

1. Igra se pokrece iz glavnog izbornika pritiskom na *Pokreni igru* – timer pocinje odmah
2. Ucitavaju se svi podaci iz SQLite baze podataka
3. Igrac i protivnik svaki vuku po 3 random karte iz spila
4. **Igracov potez:** igrac dobiva energiju i vuce kartu; klikom na gumb karte odabire koju ce odigrati; potez se automatski prebacuje ako nema energije ili ne moze platiti nijednu kartu
5. **Protivnikov potez:** protivnik automatski igra sve karte koje moze platiti; Zmaj ima 4 energije
6. Koraci 4 i 5 ponavljaju se dok HP igraca ili protivnika ne padne na 0
7. Nakon pobjede igrac odabire jednu nagradu: **+10 max HP** ili **+1 max energija**
8. Igraju se 3 rounda: *Goblin* (easy), *Vitez* (medium), *Zmaj* (hard)
9. Na kraju se prikazuje ekran s rezultatom i ukupnim vremenom igranja

Zaključak

Projekt prikazuje kako se Godot Engine može spojiti s relacijskom bazom podataka kako bi se dinamički upravljalo sadržajem igre. Implementirani su mehanizmi kartaske igre – upravljanje spilom, random vucenje karata, sustav energije, naizmjenični potezi, round sistem s nagradama i praćenje vremena igranja. Cijeli GUI izrađen je dinamički kroz GDScript bez zasebnih scene datoteka, a baza podataka omogućuje lako dodavanje novih karata i protivnika bez mijenjanja koda igre.